

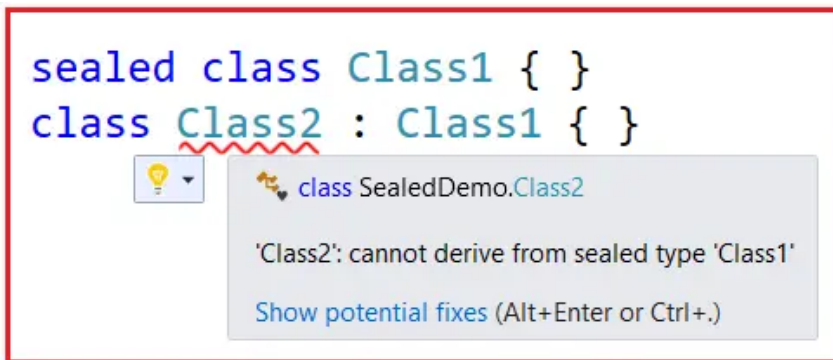
Sealed Class and Sealed Methods in C# with Examples

In this article, I am going to discuss **Sealed Class and Sealed Methods in C#** with Examples. Please read our previous article where we discussed [Partial Classes and Partial Methods in C#](#). At the end of this article, you will understand what exactly Sealed Class and Sealed Methods in C# are and when and how to use them with examples.

Sealed Class in C#

A class from which it is not possible to create/derive a new class is known as a sealed class. In simple words, we can say that when we define the class using the sealed modifier, then it is known as a sealed class and a sealed class cannot be inherited by any other classes.

To make any class a sealed class we need to use the keyword sealed. The keyword sealed tells the compiler that the class is sealed, and therefore, cannot be extended. No class can be derived from a sealed class. For a better understanding, please have a look at the below code.



Points to Remember while working with Sealed Class in C#

1. A sealed class is completely the opposite of an abstract class.
2. The sealed class cannot contain any abstract methods.
3. It should be the bottom-most class within the inheritance hierarchy.
4. A sealed class can never be used as a base class.
5. The sealed class is specially used to avoid further inheritance.
6. The keyword sealed can be used with classes, instance methods, and properties.

Note: Even if a sealed class cannot be inherited, we can still consume the class members from any other class by creating the object of the class.

Example to Understand Sealed Class in C#:

Please have a look at the following example to understand the sealed class in C#. Here, first, we created the Employee class, and then we created the Manager class inherited from the Employee class. And here we marked the Manager class as sealed and hence no further inheritance. Here, you can see while creating the TempManager class from the Manager class, we will get compile time error saying that **'TempManager': cannot derive from sealed type 'Manager'**. See, inheritance is not possible, but we can create an instance of the sealed and consume the members from outside the class. So, inside the Main method, we are creating an instance of the Manager class and invoking the two methods.

```

using System;
namespace SealedDemo
{
    public class Employee
    {
        protected int Eid, Eage;
        protected string Ename, Eaddress;
        public virtual void GetEmployeeData()
        {
            Console.WriteLine("Enter Employee Details:");
            Console.Write("Enter Employee ID:");
            Eid = int.Parse(Console.ReadLine());
            Console.Write("Enter Employee Name:");
            Ename = Console.ReadLine();
            Console.Write("Enter Employee Address:");
            Eaddress = Console.ReadLine();
            Console.Write("Enter Employee Age:");
            Eage = int.Parse(Console.ReadLine());
        }
        public virtual void DisplayEmployeeData()
        {
            Console.WriteLine("\nEmployee Details Are:");
            Console.WriteLine($"Employee ID: {Eid}");
            Console.WriteLine($"Employee Name: {Ename}");
            Console.WriteLine($"Employee Address: {Eaddress}");
            Console.WriteLine($"Employee Age: {Eage}");
        }
    }
    public sealed class Manager : Employee
    {
        double Bonus, Salary;
        public override void GetEmployeeData()
        {
            Console.WriteLine("Enter Manager Details:");
            Console.Write("Enter Manager ID:");
            Eid = int.Parse(Console.ReadLine());
            Console.Write("Enter Manager Name:");
            Ename = Console.ReadLine();
            Console.Write("Enter Manager Salary:");
            Salary = Convert.ToDouble(Console.ReadLine());
            Console.Write("Enter Manager Bonus:");
            Bonus = double.Parse(Console.ReadLine());
        }
        public override void DisplayEmployeeData()
        {
            Console.WriteLine("\nManager Details Are:");
            Console.WriteLine($"Manager ID: {Eid}");
            Console.WriteLine($"Manager Name: {Ename}");
            Console.WriteLine($"Manager Salary: {Salary}");
            Console.WriteLine($"Manager Bonus: {Bonus}");
        }
    }
}

```

```

    }
    //Further No Inheritance is Possible as we marked the class as sealed
    //public class TempManager : Manager
    //{
    //}
    //}
    class Program
    {
        static void Main(string[] args)
        {
            Manager m1 = new Manager();
            m1.GetEmployeeData();
            m1.DisplayEmployeeData();
            Console.ReadKey();
        }
    }
}

```

Output:

```

Enter Manager Details:
Enter Manager ID:1001
Enter Manager Name:Rakesh
Enter Manager Salary:100000
Enter Manager Bonus:50000

Manager Details Are:
Manager ID: 1001
Manager Name: Rakesh
Manager Salary: 100000
Manager Bonus: 50000

```

Sealed Methods in C#

The method that is defined in a parent class, if that method cannot be overridden under a child class, we call it a sealed method. That means by default, every method is a sealed method because overriding is not possible unless the method is not declared as virtual in the parent class. If a method is declared as virtual in a class, any child class of it can have the right to override that method. For example:

```

namespace SealedDemo
{
    class Parent
    {
        public virtual void Show() { }
    }
    class Child : Parent
    {
        public override void Show() { }
    }
    class GrandChild : Child

```

```

    {
        public override void Show() { }
    }
}

```

As you can see in the above code, the class Child overrides the Parent class Show method and again the GrandChild class also overrides the Show method. Now, if you want to restrict the GrandChild class not to override the Show method, then you need to use the sealed modifier along with the override modifier in the Child class. This will restrict the method to be overridden in the derived classes. For a better understanding, please have a look at the below example.

```

namespace SealedDemo
{
    class Parent
    {
        public virtual void Show() { }
    }
    class Child : Parent
    {
        public sealed override void Show() { }
    }
    class GrandChild : Child
    {
        // 'GrandChild.Show()': cannot override inherited member 'Child.Show()'
        is sealed
        public override void Show() { }
    }
}

```

So, the point that you need to remember is every method by default is sealed in C# and hence they cannot be overridden under the child class. But if you declare a method as virtual then that method can be overridden under the child classes as well as under the grandchild classes also. If you want to restrict the method, not be overridden under the grandchild classes then you need to make the method sealed inside the child class. So, the point that you need to remember is that if you want to declare a method as sealed, then it has to be declared as virtual in its base class.

Real-Time Example to Understand Sealed Class and Sealed Method in C#:

Now, let us understand the sealed class and sealed method with one real-time example. In the following example, the Printer is the Parent class and, in this class, we have defined two virtual methods. The Printer class Display method displays the unit with a dimension of 5×5 and the print method simply prints the same. Then the LaserJet class overrides the Display method to have the dimension of 10×10. If any class is going to inherit from the LaserJet class, then that class in our example InkJet class will have the same dimension of 10×10 and cannot implement its own, like 15×15, 16×16, or anything, because the Display method is marked as sealed. The following example code is self-explained and please go through the comment lines.

```

using System;
namespace SealedDemo
{
    public class Printer
    {

```

```

//The Printer class declaring two virtual methods
public virtual void Display()
{
    Console.WriteLine("Display Dimension: 5x5");
}
public virtual void Print()
{
    Console.WriteLine("Printer Printing...\n");
}
}
public class LaserJet : Printer
{
    //The LaserJet class Overriding the two parent class virtual methods
    //But making the Display method as sealed, so the child classes of LaserJet
    //will not override this method
    public sealed override void Display()
    {
        Console.WriteLine("Display Dimension: 10x10");
    }
    //The Print method can be override under the Child class of LaserJet
    public override void Print()
    {
        Console.WriteLine("LaserJet Printer Printing...\n");
    }
}
//The InkJet class can not override the Display Method as it is marked sealed
//LaserJet class.
//So, InkJet will have same Display feature i.e it also has "Display Dimension: 10x10".
public sealed class InkJet : LaserJet
{
    //The following method overriding will give compile time error
    //'InkJet.Display()': cannot override inherited member 'LaserJet.Display()'
    //because it is sealed
    //public override void Display()
    //{
    //    Console.WriteLine("Some Different Display Dimension");
    //}
    public override void Print()
    {
        Console.WriteLine("InkJet Printer Printing...");
    }
}
//As we marked the InkJet class as sealed so more inheritance using InkJet
//The following inheritance will give you compile time error
//'DotMatrix': cannot derive from sealed type 'InkJet'
//class DotMatrix : InkJet
//{
//    //{
//    //}
//}
class Program
{

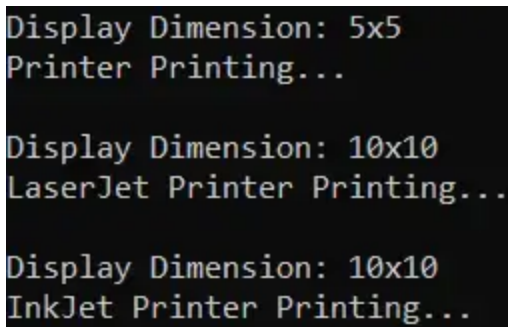
```

```

static void Main(string[] args)
{
    Printer printer = new Printer();
    printer.Display();
    printer.Print();
    LaserJet laserJet = new LaserJet();
    laserJet.Display();
    laserJet.Print();
    InkJet inkJet = new InkJet();
    inkJet.Display();
    inkJet.Print();
    Console.ReadKey();
}
}
}

```

Output:



```

Display Dimension: 5x5
Printer Printing...

Display Dimension: 10x10
LaserJet Printer Printing...

Display Dimension: 10x10
InkJet Printer Printing...

```

Note: In Inheritance, the sealed class is the bottom-most class because from this class no more inheritance is possible.

When should a method be declared as sealed in C#?

If we don't allow subclasses to override the superclass method and ensure that all sub-classes use the same superclass method logic then that method should be declared as sealed. The sealed method cannot be overridden in sub-classes violation leads to a compile-time error.

What is the difference between the private and sealed methods in C#?

The private method is not inherited whereas the sealed method is inherited but cannot be overridden in C#. So, a private method cannot be called from sub-classes whereas a sealed method can be called from sub-classes. The same private method can be defined in sub-class and it does not lead to error. For a better understanding, please have a look at the following example. The following example code is self-explained, so please go through the comment lines.

```

using System;
namespace SealedDemo
{
    public class Class1
    {

```

```

        public virtual void Method1()
        {
            Console.WriteLine("Class1 Method1");
        }
    }
    public class Class2 : Class1
    {
        //Private Method
        private void Method2()
        {
            Console.WriteLine("Class2 Private Method2");
        }
        //Sealed Method
        public sealed override void Method1()
        {
            Console.WriteLine("Class2 Sealed Method1");
        }
    }

    public class Class3 : Class2
    {
        //We cannot override Method1 because it is sealed in Class2
        //But this method is inherited and hence we can access this method
        //using Class3 instance
        //public override void Method1()
        //{
        //    Console.WriteLine("InkJet Printer Printing...");
        //}
        //Class2 Private Method Method2 is not inherited to child class and hence
        //you can define the same method here
        public void Method2()
        {
            Console.WriteLine("Class3 public Method2");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Class2 obj1 = new Class2();
            obj1.Method1();
            Class3 obj3 = new Class3();
            obj3.Method1();
            obj3.Method2();
            Console.ReadKey();
        }
    }
}

```

Output:

```
Class2 Sealed Method1  
Class2 Sealed Method1  
Class3 public Method2
```

When should a class be declared as sealed in C#?

If we want to restrict inheritance then we need to declare the class as sealed by using the sealed modifier in C#.

What are the differences between an abstract class and a sealed class in C#?

Abstract Class in C#:

1. A class that contains one or more abstract methods is known as an abstract class.
2. The abstract class can contain abstract and non-abstract methods.
3. Creating a new class from an abstract class is compulsory to consume the abstract class non-abstract members.
4. An abstract class cannot be instantiated directly; we need to create the object for its child classes to consume an abstract class.
5. We need to use the keyword `abstract` to make any class abstract.
6. An abstract class cannot be the bottom-most class within the inheritance hierarchy.

Sealed Class in C#:

1. A class from which it is not possible to derive a new class is known as a sealed class.
2. The sealed class can contain non-abstract methods; it cannot contain abstract and virtual methods.
3. It is not possible to create a new class from a sealed class.
4. We should create an object for a sealed class to consume its members.
5. We need to use the keyword `sealed` to make any class sealed.
6. The sealed class should be the bottom-most class within the inheritance hierarchy.

In the next article, I am going to discuss [Extension Methods in C#](#) with examples. Here, in this article, I try to explain **Sealed Class and Sealed Methods in C#** with examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this Sealed Class and Sealed Methods in the C# article.